

A Product Management Proposal — B2B Fintech Platform

Table of Contents

1. Understanding the Problem
 2. Improvement Proposals
 3. Collaboration with Engineering
 4. Communication & Reporting
-

Section 1: Understanding the Problem

Background

Our platform is built for finance teams at investment firms who handle daily back office operations. One of the most important features we offer is the daily transaction reconciliation tool. Finance teams use this tool to compare and match financial data coming from two places: their own internal system and data received from external sources like banks or brokers. The whole point of this exercise is to find and fix any differences before they cause problems in reporting, compliance, or fund calculations.

The tool does its job, but users have been telling us it takes too long and leads to too many errors, especially when the data is large or the differences are small. This section explains what the current experience looks like for users, where things go wrong, and what is at stake if we do not fix these problems.

What the User's Workflow Looks Like Today

Let us take Priya as an example. She is a reconciliation analyst and her day starts with downloading two data files manually. One comes from the internal ledger system and one comes from an external broker or bank. These files often look different from each other. They come in different formats, arrive at different times, and the data inside them is not always consistent.

Priya then uploads both files into the reconciliation tool and runs a comparison. The tool currently uses a simple rule: two records are only considered a match if the relevant fields are exactly the same, character by character. Once the comparison runs, Priya gets a list of what matched and what did not.

Now the real work begins. She scrolls through hundreds, sometimes thousands, of records that did not match. She compares them side by side to figure out if each one is a real problem or just a small formatting difference, like "Acme Corp" on one side and "Acme Corporation" on the other. For every decision she makes, she has to open both records, check multiple fields, decide what to do, and write down the outcome, usually in a separate spreadsheet or a notes field outside the main tool.

By mid morning she might have worked through less than half the list. The rest piles up with no priority order, no one assigned to handle specific items, and no record of what was already

looked at. Her team lead has no visibility into how much is left or how serious the remaining items are.

Pain Points and Friction Users Experience

The matching logic is too strict and creates fake problems

Because the tool only accepts exact matches, small differences in text show up as mismatches even when the transaction is clearly the same. A counterparty name written slightly differently, a reference number with an extra zero, or an amount rounded to a different decimal place, all of these create extra work for no real reason. Analysts end up spending most of their time on things that are not real problems.

There is no way to tell what is urgent and what is not

When 200 exceptions appear in a flat list, they all look the same. A transaction with a 900,000 dollar discrepancy sits next to one with a 12 dollar rounding difference. Analysts have to manually sort through everything each morning just to figure out where to start, and that itself takes time.

Teams cannot work together on the same file

When more than one analyst is working on the same reconciliation, there is no way to assign items, track who is handling what, or stop two people from investigating the same record at the same time. This leads to wasted effort and things falling through the cracks.

There is no record of why a decision was made

When an analyst approves a match or closes an exception, nothing is recorded about why that decision was made. There is no log, no rule reference, no name attached to it. For firms that need to comply with financial regulations or go through audits, this is a serious gap because auditors have to piece together decisions from emails and handwritten notes.

Settlement timing differences look like real errors

Sometimes a transaction appears in the bank feed one day but is recorded internally the next day. This is completely normal in finance and is called a settlement lag. But the current tool treats it like a genuine mismatch. Analysts have to manually close these items every single day even though they are not real problems.

Business and User Outcomes at Risk

The impact of a poor reconciliation experience goes well beyond daily frustration. Research across the finance industry shows that 89 percent of finance leaders say they regularly make monthly decisions based on data that is incomplete or wrong, and 55 percent say they do not have the visibility they need to guide their organization well.

For an investment firm, the stakes are even higher. One missed or wrongly classified transaction can lead to errors in fund value calculations, incorrect profit and loss reports, regulatory filing failures, or wrong numbers on investor statements. These are not just internal problems. They carry financial penalties and damage the firm's reputation.

From the user's side, when analysts cannot trust the tool to show them real problems clearly, they start checking everything by hand. This defeats the purpose of having the software in the first place. The daily pressure also increases the risk of burnout in teams that are already working lean.

Section 2: Improvement Proposals

Three improvements are proposed below. Each one targets a different part of the reconciliation experience. They are designed to be built in phases. The first improvement has the biggest immediate impact and also sets the foundation for the other two.

Improvement 1: Smart Matching Engine with Confidence Scores

What the change is

We replace the current strict matching logic with a smarter, rules based engine that looks at multiple fields to decide whether two records are likely the same transaction. It checks the amount, the date (with a small window of plus or minus one day), the reference number, the counterparty name using flexible text matching to handle small differences, and the currency. Each suggested match gets a score between 0 and 100, and a short plain English explanation tells the analyst exactly why the match was suggested.

Matches are split into three groups based on the score:

Scores between 95 and 99 are shown at the top and can be approved with a single click or all at once in bulk.

Scores between 85 and 94 are shown for individual review with the scoring breakdown visible so the analyst understands the reasoning.

Scores below 85 go directly into the exception queue for deeper investigation.

Why it is helpful

Flexible text matching handles the small differences that cause most of the false breaks today, things like abbreviations, typos, and minor formatting inconsistencies. The confidence score gives analysts enough information to make a quick decision without needing to open every record individually. Over time, as analysts approve and reject suggestions, the engine can get better at matching and reduce the workload further.

Leading reconciliation platforms that use this kind of matching report auto match rates above 90 percent, which means analysts spend their time on real problems instead of noise.

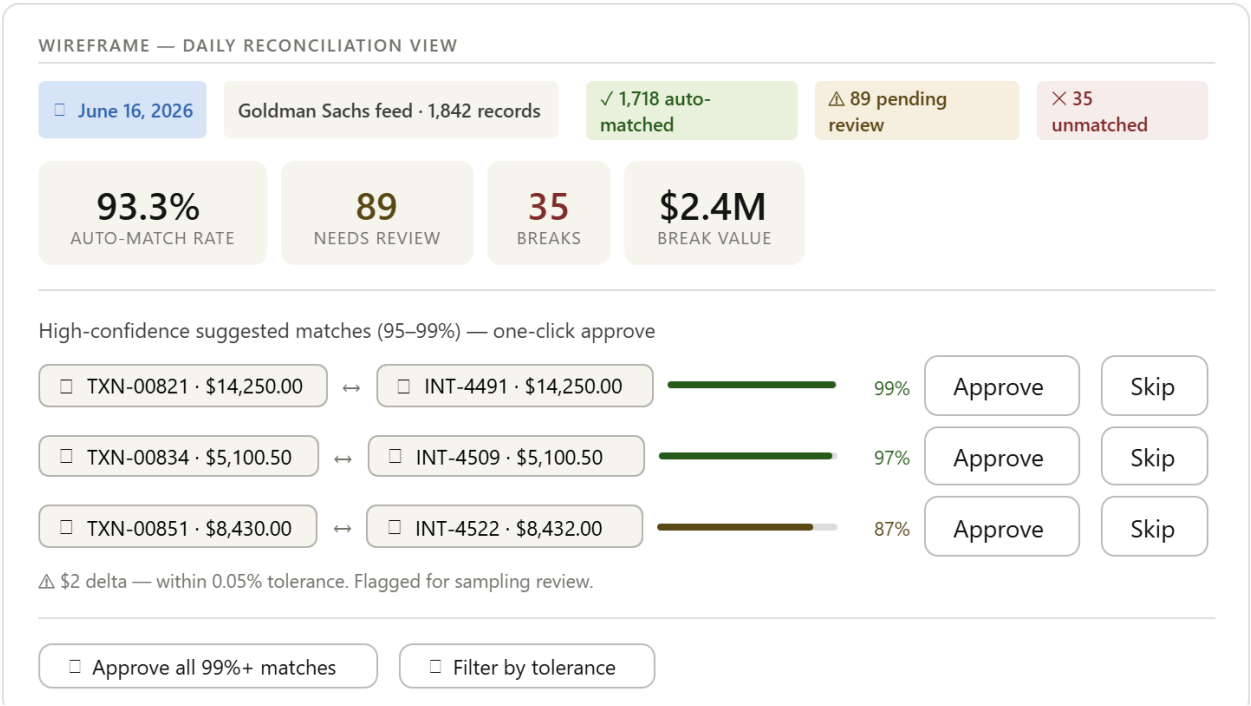
Wireframe description

The main view shows a summary bar at the top with the date, the data source name, and four key numbers: how many records were auto matched, how many need review, how many are unmatched, and the total value of the breaks. Below that, a list shows each suggested match as a row. On the left is the external transaction, on the right is the internal one, and in the middle is a horizontal bar showing the confidence score as a percentage. Each row has an Approve button and a Skip button. At the bottom, a button lets the analyst approve all matches scoring 99 percent or above in one click. For matches that include a small amount difference, a short note appears underneath the row explaining whether it falls within the accepted tolerance.

User story

As Priya, when I open the tool, I want to see all the obvious matches already sorted and ready for quick approval so I can focus my time only on the records that are genuinely unclear, not on re reading transactions that are clearly the same thing with a slightly different name.

IMPROVEMENT 1 — SMART MATCHING PANEL WITH CONFIDENCE SCORES



Improvement 2: Exception Triage Queue with Priority Routing

What the change is

Instead of showing all unresolved items in a flat list, we build a structured exception panel that automatically sorts and groups items by type and urgency. Each exception gets a category:

amount mismatch, missing record, timing difference, or duplicate. It also gets a priority level based on the size of the break and how long it has been open. Each item can be assigned to a specific team member and comes with a timer showing how long is left before it needs to be resolved.

Items that are only flagged because of a one day settlement lag are automatically tagged for auto close using a predefined rule. Analysts can confirm the rule applied correctly through a quick sampling check rather than manually closing each item one by one.

Why it is helpful

When exceptions are sorted by size and type, analysts immediately know where to focus. The biggest financial risks get addressed first. Assignment and deadline tracking add accountability and give team leads a live view of progress without needing to run a status meeting. The auto close rule for timing differences alone can remove a significant chunk of the daily exception list for any firm that deals with standard settlement lags.

Wireframe description

The top of the panel shows three tabs with total break values: High Priority, Medium, and Timing or Low. Below that, a sortable table lists each exception with a short ID, a plain English description of the issue, a priority badge, the break amount, how many days it has been open, and action buttons to either investigate or auto close. A progress bar at the bottom shows how many exceptions the team has resolved that day. Buttons in the footer allow the team lead to assign items, snooze timing items in bulk, or export a report.

IMPROVEMENT 2 — EXCEPTION TRIAGE QUEUE WITH PRIORITY ROUTING

WIREFRAME — EXCEPTION MANAGEMENT PANEL

High priority · \$1.2M+ break value

Medium · \$180K

Low / timing · \$42K

EXC-0041	TXN-00812 — Present in bank, missing in internal ledger	High	\$980,000	0d	Investigate
EXC-0044	TXN-00829 — Amount mismatch: bank \$250K vs internal \$248K	Med	\$2,000	1d	Investigate
EXC-0047	TXN-00841 — Timing diff: bank T+0, internal T+1 (settlement lag)	Timing	\$75,000	0d	Auto-close
EXC-0049	TXN-00858 — Duplicate detected in broker feed	Med	\$14,250	0d	Investigate

Assigned to: Priya Mehta · SLA: resolve within 4h · Progress

22 of 35 exceptions resolved today

Assign

Snooze timing items

Export report

User story

As a reconciliation team lead, I want to see all open exceptions sorted by how much money is at risk so I can assign the most serious items to senior analysts before 10 AM, and I want the system to handle the routine timing items automatically so my team is spending their time where it actually matters.

Improvement 3: Audit Trail and Match Explainability

What the change is

Every match decision, whether it was made by the system automatically or approved manually by an analyst, creates a structured log entry. The log records who made the decision, when it happened, which matching rule was applied, and what the score breakdown looked like. A side by side view of the two transactions shows exactly which fields matched, which ones were different, and whether any tolerance rule was used to allow the match.

This log can be exported as a PDF or spreadsheet and is directly accessible to compliance officers and external auditors from within the platform.

Why it is helpful

When a system shows only a score without explaining the reasoning, it is harder to trust and harder to audit. For investment firms operating under financial regulations, being able to show exactly why a match was approved is not a nice to have feature. It is often a requirement. This improvement also saves analysts hours of time during audit periods because the evidence is already organized and ready to share, rather than scattered across emails and spreadsheets.

Wireframe description

The detail view shows both transaction records side by side, field by field. Fields that match appear normally. Fields that differ are highlighted in amber. Below the comparison, a box shows the scoring explanation in plain language, for example: Reference number matched (40 points), Date matched (25 points), Counterparty name was very similar using flexible matching (18 points), Amount difference of 2 dollars falls within the 0.05 percent tolerance (4 points), giving a total of 87 out of 100. Below that, a timeline shows each action taken on the record with timestamps, the name of the person who acted, and any notes they added. Footer buttons allow the analyst to approve the match, reject and escalate it, add a comment, or export the audit log.

User story

As a compliance officer, when an auditor asks why two records were matched despite a small amount difference, I want to open the audit log, see that the analyst approved it under a specific rounding tolerance rule, and export that record as a file in one click, without having to contact anyone or search through old emails.

IMPROVEMENT 3 — AUDIT TRAIL & EXPLAINABILITY PANEL

WIREFRAME — MATCH DETAIL & AUDIT LOG (CLICK ANY MATCH TO EXPAND)

BANK FEED — TXN-00851

Date	16 Jun 2026
Amount	\$8,432.00
Reference	GS-INV-2048
Counterparty	Acme Corp
Currency	USD

↔

INTERNAL LEDGER — INT-4522

Date	16 Jun 2026
Amount	\$8,430.00
Reference	GS-INV-2048
Counterparty	Acme Corporation
Currency	USD

Why matched at 87%: Reference identical (40pts) · Date match (25pts) · Counterparty fuzzy "Acme Corp" ≈ "Acme Corporation" (18pts) · Amount delta \$2.00 within 0.05% tolerance (4pts) · Total: 87/100

Audit trail

09:14:02	System	Auto-suggested match at 87% confidence
09:21:38	P. Mehta	Opened for review — noted \$2 rounding discrepancy
09:24:11	P. Mehta	Approved match — delta within FX rounding tolerance. Rule: R-14

☐ Approve match

☐ Reject & escalate

☐ Add comment

☐ Export audit log

Section 3: Collaboration with Engineering

For this section the focus is on Improvement 1, the Smart Matching Engine, because it has the biggest impact and builds the foundation that the other two features depend on.

How I Would Work with a Developer

I would start with a working session together, not a handoff of a finished document. The goal of that first meeting is to understand what data we actually have before we decide how to build anything. I would bring the wireframes and a few real world examples of the kind of mismatches users deal with every day, with names anonymized. The engineer brings knowledge of how the current matching logic works, what the data pipeline looks like, and any existing documentation about the data structure.

From there we would figure out the matching rules together: which fields are available in both data sources, which ones are reliably filled in, and how much weight each one should carry in the scoring. We would sketch the model out together and agree on what the three score tiers should be before anyone writes a single line of code.

I would use a short brief of one to two pages instead of a long requirements document for the first phase, with clear acceptance criteria written up front so both sides know exactly what we are building and what done looks like.

Technical Questions I Would Ask

1. Is the data from both sources already normalized into a common structure before matching happens, or are we comparing raw files that might be formatted completely differently? If normalization does not exist yet, that needs to be sorted out before the matching engine can work properly.
 2. What is the highest number of transactions we need to handle in a single reconciliation run, and how fast does the matching need to finish? Comparing every record in one file against every record in another gets very expensive at large scale. Good reconciliation platforms handle this by pre grouping records into smaller candidate sets using things like amount ranges or date windows before running the detailed comparison, and we would need to decide if that approach makes sense for us.
 3. Can we store the decisions analysts make, meaning which suggestions they approved and which they rejected, so we can use that data later to improve the scoring weights over time?
 4. Does the current data pipeline support situations where one external record needs to match against multiple internal records, for example when one broker payment covers a batch of smaller internal transactions?
 5. Who is responsible for defining the tolerance thresholds, meaning how big an amount difference is acceptable before we stop suggesting a match, and do those thresholds change depending on the type of transaction or the counterparty?
-

Edge Cases and Data Issues to Clarify

Some data problems can cause a matching engine to produce wrong results without showing any obvious error. These need to be understood and planned for before we build anything.

Two different transactions that look identical: If two separate transactions have the same amount, the same date, and the same counterparty, the engine might suggest them as a match when they should not be. These cases need a human confirmation step and should never be auto approved.

Time zone differences: If the bank records timestamps in one time zone and the internal system uses another, a transaction late in the evening in one system might appear as a different calendar date in the other. Without fixing this first, the engine will generate a large number of false timing breaks.

One external record matching multiple internal records: Batch settlements where a single bank entry covers several smaller internal transactions are common. The current one to one model does not handle this and we need to decide how the engine will behave in these situations.

Duplicate records in the feed: Some broker feeds occasionally send the same transaction twice with slightly different reference numbers. The engine needs to catch these and flag them as duplicates rather than treating them as two separate match candidates.

Different decimal precision: If one system stores amounts to two decimal places and the other stores them to four, the same transaction might look like a mismatch. We need an agreed rounding rule applied before matching runs.

Blank reference fields: When the reference number is missing in one of the records, the engine should still be able to produce a reasonable score based on the other fields rather than immediately scoring the match very low.

Ensuring the Feature is Testable and Delivered with Confidence

Before we switch anything over in production, I would suggest running the new engine in parallel with the existing process for two full reconciliation cycles. During that time, both systems run at the same time and we compare their outputs. Where did the engine suggest a match that the analyst confirmed as correct? Where did it suggest something the analyst rejected? Where did it miss a match that the analyst found manually?

This gives us three numbers to track:

The auto match rate, meaning what percentage of records the engine matched correctly without human input. We would aim for 90 percent or above.

The false positive rate, meaning how often the engine suggests a match that turns out to be wrong. We would want this below 0.1 percent because a wrong auto match is worse than a missed match.

The false negative rate, meaning how often the engine misses a real match. We track this but focus on the false positive rate first during early rollout.

The acceptance criteria would be written down and agreed with the engineering lead and QA before the sprint starts, not after. We would also set up a feature flag so the engine can be switched off instantly if something unexpected comes up in production. After the first two live cycles, I would schedule a short review to look at real match data and adjust any thresholds that need tuning based on what we actually see.

Section 4: Communication & Reporting

Internal Slack Message

Channel: #product-recon-improvements

From: [Product Manager Name]

Date: June 16, 2026

Hey team, sharing a quick update on the reconciliation tool review and what we are planning to do

The problem

Users are spending too much time on manual matching because the tool flags minor text differences as full mismatches. On top of that, all exceptions look equally urgent regardless of the break value, and there is no audit trail when an analyst makes a decision. This is slowing teams down and becoming a compliance risk.

What we are proposing

Three improvements in order of priority:

- A smart matching engine with confidence scores so high confidence matches can be approved in bulk
- An exception triage queue that ranks breaks by size and auto closes routine timing items
- An audit trail that logs every match decision for compliance and audit purposes

Next steps

- Eng sync Thursday to review data schema and matching feasibility @dev-lead can you confirm?
- Design wireframe review Monday @design-lead sharing the file with your team.
- Acceptance criteria draft from me by end of this week

Full proposal available if anyone wants a deeper walkthrough. Drop questions here or ping me directly. If any team has any high priority task scheduled on Thursday, kindly let me know. We shall reschedule the call.

A Note on Where We Are Headed Next

Once the first phase is live and we have real match data coming through, we will have what we need to start thinking about automation more seriously. The scoring model and the audit logs we build in phase one become the training material for a smarter system that can continuously improve on its own without needing manual rule updates.

Research from across the industry shows that adding machine learning to a rules based reconciliation engine can free up to 50 percent of operations team time for more valuable analytical work. The investment we make in explainability and audit trails now is not just about compliance. It is what makes automated reconciliation trustworthy enough to use in a regulated financial environment later.